POLITÉCNICO GRANCOLOMBIANO

Facultad de Ingeniería, Diseño e Innovación

Módulo

ÉNFASIS PROFESIONAL I — INTEGRACIÓN CONTINUA

Proyecto de software basado en herramientas de integración continua

ENTREGA 1 · SEMANA 3 · ESCENARIO 3

Construcción de dos contenedores Docker comunicados entre sí

Proyecto: Gestor de Tareas (API REST + base de datos)

Dato	Contenido
Integrantes del grupo	[Nombre completo y código de cada integrante]
Grupo / Subgrupo	[Número de subgrupo]
Programa académico	[Programa]
Docente	[Nombre del docente]
Repositorio GitHub	[https://github.com/usuario/entrega1-integracion-continua]
Fecha de entrega	07 de noviembre de 2026

Contenido

1. Introducción
2. Objetivos
3. Descripción del proyecto de software
4. Repositorio en GitHub
5. Arquitectura de los dos contenedores
6. Contenedor 1 — Servicio web (Flask)
7. Contenedor 2 — Base de datos (PostgreSQL)
8. Orquestación y comunicación entre contenedores
9. Puesta en marcha
10. Evidencias de ejecución
11. Integración continua (pipeline en GitHub Actions)
12. Dificultades encontradas y soluciones
13. Roles y responsabilidades del equipo
14. Conclusiones
15. Referencias

1. Introducción

La integración continua es una práctica de la ingeniería de software que consiste en integrar el trabajo de todos los miembros de un equipo en un repositorio compartido de forma frecuente, verificando cada integración mediante construcciones y pruebas automatizadas. Su propósito es detectar los errores lo antes posible, reducir el riesgo de las entregas y mantener el software en un estado siempre desplegable.

Para que esa práctica sea viable, el software debe ejecutarse de la misma manera en el equipo de cada desarrollador, en el servidor de integración y en el entorno de producción. La contenerización resuelve ese problema: Docker empaqueta la aplicación junto con su sistema operativo base, sus dependencias y su configuración en una unidad portable e inmutable.

El presente documento corresponde a la Entrega 1 (Semana 3 — Escenario 3) del proyecto del módulo. De acuerdo con la guía del proyecto, en este punto el proyecto ya se encuentra creado en un repositorio GitHub y se emplea Docker para construir dos contenedores que deben estar comunicados entre sí. El trabajo realizado se documenta a continuación, acompañado de las evidencias obtenidas durante la ejecución real del entorno.

2. Objetivos

2.1 Objetivo general

Construir, mediante Docker, una plataforma de software compuesta por dos contenedores que se comuniquen entre sí, alojada en un repositorio de GitHub, como base para la aplicación posterior de herramientas de integración continua.

2.2 Objetivos específicos

- Crear el repositorio del proyecto en GitHub y definir una estrategia de ramas que soporte la integración continua.
- Desarrollar una aplicación de software funcional que sirva como caso de estudio del módulo.
- Construir un contenedor para el servicio de aplicación a partir de un Dockerfile propio.
- Desplegar un segundo contenedor con el motor de base de datos a partir de una imagen oficial.
- Establecer la comunicación entre ambos contenedores mediante una red definida por el usuario y la resolución DNS interna de Docker.
- Orquestar el ciclo de vida de los dos contenedores con Docker Compose.
- Documentar y evidenciar el funcionamiento de la comunicación entre los contenedores.

3. Descripción del proyecto de software

Gestor de Tareas: una API REST que permite registrar, consultar y eliminar las tareas de un equipo de desarrollo. Se escogió este caso por tres razones:

- Es lo suficientemente pequeño para que el foco del módulo siga estando en las herramientas de integración continua y no en la lógica de negocio.
- Exige de forma natural dos componentes separados —un servicio de aplicación y un motor de datos persistente—, que es justamente lo que pide la entrega.
- Permite escribir pruebas automatizadas sencillas, necesarias para las entregas siguientes con Jenkins y Travis CI.

La aplicación expone los siguientes recursos:

Método	Ruta	Descripción
GET	/	Identificación del servicio y de su destino de base de datos
GET	/health	Verifica la comunicación con el contenedor de base de datos
GET	/api/tareas	Lista todas las tareas almacenadas
GET	/api/tareas/<id>	Consulta una tarea por su identificador
POST	/api/tareas	Crea una nueva tarea
DELETE	/api/tareas/<id>	Elimina una tarea

3.1 Tecnologías empleadas

Componente	Tecnología	Versión
Lenguaje de la aplicación	Python	3.12
Framework web	Flask	3.0.3
Servidor de aplicación	Gunicorn	22.0.0
Controlador de base de datos	psycopg2-binary	2.9.9
Motor de base de datos	PostgreSQL (Alpine)	16
Contenerización	Docker Engine	29.7.2
Orquestación local	Docker Compose	v2
Control de versiones	Git / GitHub	—
Pruebas automatizadas	pytest	8.2.2
Integración continua	GitHub Actions	—

4. Repositorio en GitHub

El código fuente completo del proyecto se encuentra versionado en GitHub. El repositorio contiene tanto la aplicación como los archivos de contenerización, de manera que cualquier integrante del equipo —o el servidor de integración continua— pueda reproducir el entorno con un único comando.

URL del repositorio: *[pegar aquí la URL del repositorio del grupo]*

4.1 Estructura del repositorio

```
entrega1-integracion-continua/
|
+-- app/                          CONTENEDOR 1 - servicio web
|   +-- app.py                    API REST en Flask
|   +-- requirements.txt          dependencias de Python
|   +-- Dockerfile                construccion de la imagen del servicio web
|   +-- .dockerignore
|   +-- tests/
|       +-- test_app.py           pruebas unitarias
|
+-- db/                           CONTENEDOR 2 - base de datos
|   +-- init.sql                  creacion de la tabla y datos iniciales
|
+-- .github/
|   +-- workflows/
|       +-- ci.yml                pipeline de integracion continua
|
+-- evidencias/                   capturas y salidas de la ejecucion
+-- docs/                         documento de la entrega
+-- docker-compose.yml            orquestacion de los dos contenedores
+-- env.example                   plantilla de variables de entorno
+-- .gitignore
+-- README.md
```

4.2 Estrategia de ramas

Se adoptó un flujo de trabajo basado en ramas que hace posible la integración continua:

Rama	Propósito	Regla
main	Versión estable del proyecto	Solo recibe cambios mediante pull request aprobado
develop	Integración del trabajo del equipo	Punto de partida de las ramas de funcionalidad
feature/<nombre>	Trabajo individual de cada integrante	Se fusiona en develop mediante pull request

Cada push y cada pull request sobre main y develop dispara automáticamente el pipeline descrito en la sección 11, que debe terminar en verde antes de permitir la fusión.

4.3 Manejo de credenciales

Las credenciales de la base de datos no se escriben en el código ni se suben al repositorio. Se gestionan mediante variables de entorno: el archivo env.example documenta cuáles son necesarias y el archivo .env real está excluido en .gitignore. Si el archivo .env no existe, docker-compose.yml aplica valores por defecto aptos únicamente para el entorno local de desarrollo.

5. Arquitectura de los dos contenedores

La plataforma está compuesta por dos contenedores que se ejecutan de forma aislada y que se comunican exclusivamente a través de una red interna de Docker creada para el proyecto.

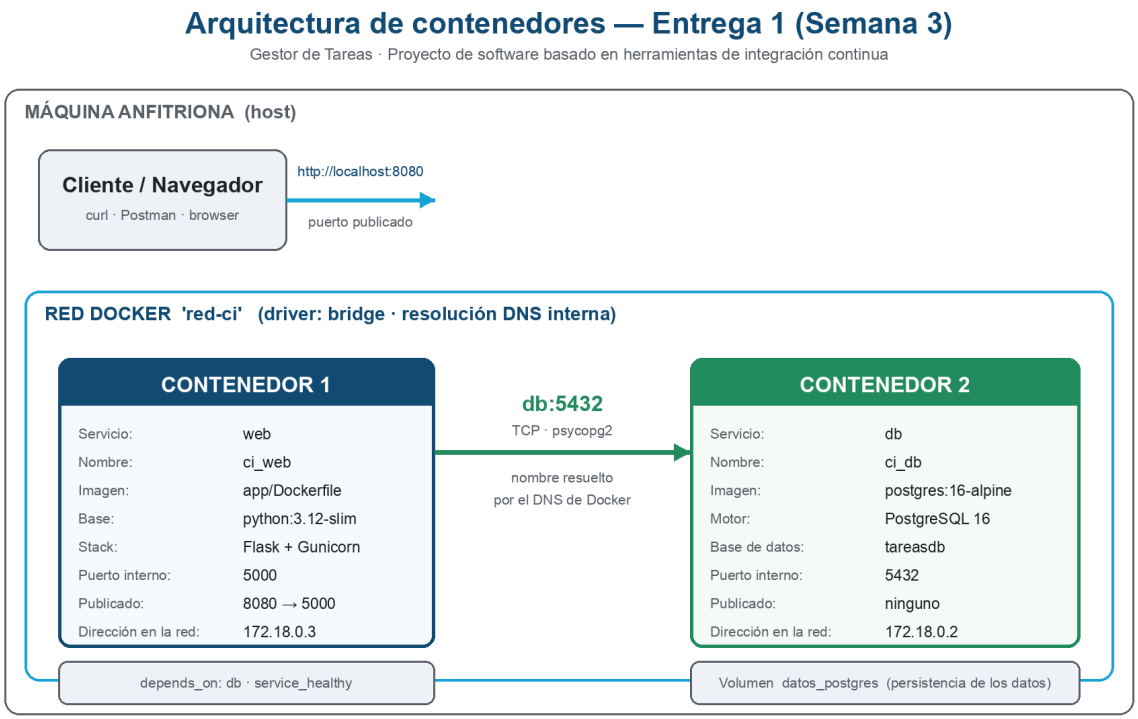


Figura 1. Arquitectura de contenedores del proyecto.

Característica	Contenedor 1	Contenedor 2
Servicio en Compose	web	db
Nombre del contenedor	ci_web	ci_db
Función	Servicio de aplicación (API REST)	Persistencia de datos
Imagen	Construida con app/Dockerfile	postgres:16-alpine (oficial)
Imagen base	python:3.12-slim	Alpine Linux
Puerto interno	5000	5432

Puerto publicado en el host	8080	Ninguno
Dirección en la red red-ci	172.18.0.3	172.18.0.2
Volumen	Ninguno	datos_postgres

Obsérvese que el contenedor de base de datos no publica ningún puerto hacia la máquina anfitriona. Esta decisión es deliberada: garantiza que el único camino posible hacia PostgreSQL sea la red interna de Docker, lo que a la vez constituye una prueba de que la comunicación ocurre efectivamente entre contenedores y no a través del host.

6. Contenedor 1 — Servicio web (Flask)

Este contenedor se construye a partir de un Dockerfile propio. El archivo aplica varias buenas prácticas: imagen base ligera y oficial, instalación de dependencias antes de copiar el código para aprovechar la caché de capas, ejecución con un usuario sin privilegios, verificación de salud integrada y uso de un servidor WSGI de producción en lugar del servidor de desarrollo de Flask.

app/Dockerfile

```
# ==============================================================================
# CONTENEDOR 1 - Servicio web (API REST en Flask)
# Proyecto de software basado en herramientas de integracion continua
# ==============================================================================

# Imagen base ligera y oficial de Python
FROM python:3.12-slim

# Metadatos de la imagen
LABEL proyecto="Gestor de Tareas - Integracion Continua"
LABEL institucion="Politecnico Grancolombiano"
LABEL contenedor="web"

# Evita archivos .pyc y fuerza salida sin buffer (logs en tiempo real)
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

# Directorio de trabajo dentro del contenedor
WORKDIR /usr/src/app

# Dependencias del sistema necesarias para psycopg2 y para el healthcheck
RUN apt-get update \
    && apt-get install -y --no-install-recommends curl libpq5 \
    && rm -rf /var/lib/apt/lists/*

# Se copian primero las dependencias para aprovechar la cache de capas de Docker
COPY requirements.txt .
RUN pip install --no-cache-dir --upgrade pip \
    && pip install --no-cache-dir -r requirements.txt

# Se copia el codigo fuente de la aplicacion
COPY . .

# Usuario sin privilegios (buena practica de seguridad)
RUN useradd --create-home appuser && chown -R appuser:appuser /usr/src/app
USER appuser

# Puerto que expone el contenedor
EXPOSE 5000

# Verificacion de salud del contenedor
HEALTHCHECK --interval=15s --timeout=5s --start-period=20s --retries=5 \
    CMD curl -f http://localhost:5000/health || exit 1
```

```
# Servidor de produccion WSGI
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "--access-logfile", "-", "app:app"]
```

6.1 Conexión hacia el contenedor de base de datos

El punto central de la entrega es cómo el código localiza al otro contenedor. La aplicación no utiliza una dirección IP fija ni localhost, sino el nombre del servicio declarado en docker-compose.yml, que la red de Docker resuelve por DNS:

Fragmento de app/app.py

```
DB_HOST = os.getenv("DB_HOST", "db")     # <-- nombre del servicio en Compose
DB_PORT = os.getenv("DB_PORT", "5432")
DB_NAME = os.getenv("DB_NAME", "tareasdb")
DB_USER = os.getenv("DB_USER", "ciuser")
DB_PASSWORD = os.getenv("DB_PASSWORD", "cipassword")


def get_connection(reintentos: int = 10, espera: int = 3):
    """Abre conexion con el contenedor de PostgreSQL."""
    ultimo_error = None
    for intento in range(1, reintentos + 1):
        try:
            return psycopg2.connect(
                host=DB_HOST, port=DB_PORT, dbname=DB_NAME,
                user=DB_USER, password=DB_PASSWORD, connect_timeout=5,
            )
        except psycopg2.OperationalError as exc:
            ultimo_error = exc
            time.sleep(espera)
    raise ultimo_error
```

Usar localhost sería un error frecuente: dentro de un contenedor, localhost apunta al propio contenedor y no a la máquina anfitriona ni a los demás contenedores. La función incorpora además una política de reintentos, porque el servicio web puede quedar listo antes de que el motor de base de datos termine de inicializarse.

7. Contenedor 2 — Base de datos (PostgreSQL)

El segundo contenedor utiliza la imagen oficial postgres:16-alpine. No requiere un Dockerfile propio: toda su configuración se realiza mediante variables de entorno y mediante el script de inicialización que se monta en el directorio /docker-entrypoint-initdb.d/, que la imagen oficial ejecuta automáticamente la primera vez que arranca.

db/init.sql

```
-- ============================================================================
-- CONTENEDOR 2 - Base de datos PostgreSQL
-- Script de inicializacion: se ejecuta automaticamente la primera vez que
-- arranca el contenedor (se monta en /docker-entrypoint-initdb.d/).
-- ============================================================================

CREATE TABLE IF NOT EXISTS tareas (
    id          SERIAL PRIMARY KEY,
    titulo      VARCHAR(150) NOT NULL,
```

```
    descripcion TEXT         DEFAULT '',
    estado      VARCHAR(30)  NOT NULL DEFAULT 'pendiente',
    responsable VARCHAR(100) DEFAULT 'sin asignar',
    creada_en   TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX IF NOT EXISTS idx_tareas_estado ON tareas (estado);

-- Datos de prueba para evidenciar la comunicacion entre contenedores
INSERT INTO tareas (titulo, descripcion, estado, responsable) VALUES
    ('Crear repositorio en GitHub',
     'Inicializar el repositorio remoto del proyecto y definir ramas main y develop',
     'completada', 'Equipo'),
    ('Construir contenedor de aplicacion',
     'Escribir el Dockerfile del servicio web en Flask',
     'completada', 'Equipo'),
    ('Construir contenedor de base de datos',
     'Configurar la imagen de PostgreSQL y el script de inicializacion',
     'completada', 'Equipo'),
    ('Verificar comunicacion entre contenedores',
     'Probar el endpoint /health y la consulta a la tabla tareas',
     'en progreso', 'Equipo'),
    ('Preparar entrega de la semana 3',
     'Documentar arquitectura, evidencias y conclusiones',
     'pendiente', 'Equipo');
```

Los datos se almacenan en el volumen con nombre datos_postgres, de modo que la información sobrevive a la eliminación y recreación del contenedor.

8. Orquestación y comunicación entre contenedores

Ambos contenedores se definen y se gobiernan desde un único archivo docker-compose.yml. En él se declara la red red-ci, a la que se conectan los dos servicios, y se establece el orden de arranque.

docker-compose.yml

```
# ============================================================================
# Orquestacion de los DOS CONTENEDORES del proyecto
#
#   web  -> Contenedor 1: API REST en Flask (construido desde ./app)
#   db   -> Contenedor 2: PostgreSQL 16 (imagen oficial)
#
# Ambos se conectan a la red definida por el usuario 'red-ci'. Docker provee
# resolucion DNS interna dentro de esa red: el contenedor 'web' alcanza al
# contenedor 'db' usando el nombre de servicio 'db' como nombre de host.
# ============================================================================

services:

  # --------------------------- CONTENEDOR 2 -------------------------------
  db:
    image: postgres:16-alpine
    container_name: ci_db
    restart: unless-stopped
    environment:
      POSTGRES_DB: ${DB_NAME:-tareasdb}
      POSTGRES_USER: ${DB_USER:-ciuser}
      POSTGRES_PASSWORD: ${DB_PASSWORD:-cipassword}
    volumes:
      # Persistencia de los datos aunque se elimine el contenedor
      - datos_postgres:/var/lib/postgresql/data
      # Script que crea la tabla y carga los datos iniciales
      - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    networks:
      - red-ci
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER:-ciuser} -d ${DB_NAME:-tareasdb}"]
```

```
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 10s

  # ---------------------------- CONTENEDOR 1 -------------------------------
  web:
    build:
      context: ./app
      dockerfile: Dockerfile
    container_name: ci_web
    restart: unless-stopped
    environment:
      DB_HOST: db          # <-- nombre del servicio, resuelto por la red Docker
      DB_PORT: 5432
      DB_NAME: ${DB_NAME:-tareasdb}
      DB_USER: ${DB_USER:-ciuser}
      DB_PASSWORD: ${DB_PASSWORD:-cipassword}
    ports:
      # host:contenedor -> la API queda disponible en http://localhost:8080
      - "8080:5000"
    depends_on:
      db:
        condition: service_healthy   # el web arranca cuando la BD esta lista
    networks:
      - red-ci

# Red propia del proyecto (aislada del resto de contenedores del host)
networks:
  red-ci:
    driver: bridge

# Volumen con nombre para la persistencia de PostgreSQL
volumes:
  datos_postgres:
```

8.1 Mecanismo de comunicación

La comunicación entre los dos contenedores se apoya en tres elementos:

- **Red definida por el usuario.** Al crear una red definida por el usuario con el driver bridge, Docker habilita en ella un servidor DNS interno.
- **Resolución por nombre de servicio.** Ese servidor DNS resuelve el nombre de cada servicio a la dirección IP que el contenedor tiene dentro de la red. Por eso DB_HOST vale db y no una dirección numérica: si el contenedor se recrea y cambia de IP, el nombre sigue siendo válido.
- **Arranque ordenado por estado de salud.** La combinación de depends_on con condition: service_healthy y el healthcheck de PostgreSQL (pg_isready) hace que el contenedor web solo arranque cuando la base de datos está realmente lista para aceptar conexiones, y no simplemente cuando el proceso ha sido lanzado.

9. Puesta en marcha

El entorno completo se levanta con un solo comando, lo que constituye uno de los beneficios directos de la contenerización para la integración continua: cualquier integrante del equipo, o el servidor de CI, obtiene exactamente el mismo entorno.

```
# 1. Clonar el repositorio
git clone <URL-DEL-REPOSITORIO>
```

```
cd entrega1-integracion-continua

# 2. (Opcional) crear el archivo de variables de entorno
cp env.example .env

# 3. Construir las imagenes y levantar los dos contenedores
docker compose up --build -d

# 4. Comprobar el estado
docker compose ps

# 5. Detener el entorno
docker compose down        # conserva el volumen de datos
docker compose down -v     # elimina tambien el volumen
```

10. Evidencias de ejecución

Las siguientes evidencias se obtuvieron ejecutando el entorno de forma real. El archivo completo de salidas se adjunta en evidencias/01-salidas-consola.txt.

10.1 Los dos contenedores en ejecución

```
$ docker compose ps
NAME     IMAGE                                SERVICE  STATUS                    PORTS
ci_db    postgres:16-alpine                   db       Up 18 seconds (healthy)  5432/tcp
ci_web   entrega1-integracion-continua-web   web      Up 12 seconds (healthy)
0.0.0.0:8080->5000/tcp
```

Ambos contenedores se encuentran en estado healthy. El contenedor ci_web publica el puerto 8080 del host; el contenedor ci_db expone el 5432 únicamente dentro de la red interna.

10.2 Red compartida y direcciones asignadas

```
$ docker network ls | grep red-ci
b24872452656   entrega1-integracion-continua_red-ci   bridge    local

$ docker network inspect entrega1-integracion-continua_red-ci
ci_db  -> 172.18.0.2/16
ci_web -> 172.18.0.3/16
```

10.3 Resolución DNS: el contenedor web localiza al contenedor db

```
$ docker compose exec web getent hosts db
172.18.0.2      db

$ docker compose exec web python
>>> socket.gethostbyname('db')
IP resuelta para db: 172.18.0.2
```

El nombre del servicio db se resuelve a la dirección del contenedor de base de datos dentro de la red red-ci.

10.4 Prueba de comunicación: endpoint /health

```
$ curl -s http://localhost:8080/health
{
    "comunicacion_contenedores": "establecida",
    "estado": "ok",
    "host_base_datos": "db",
    "motor": "PostgreSQL 16.15 on aarch64-unknown-linux-musl, compiled by
              gcc (Alpine 15.2.0) 15.2.0, 64-bit"
}
```

El contenedor web consultó la versión del motor al contenedor db y obtuvo respuesta: la comunicación entre los dos contenedores está establecida.

10.5 Lectura de datos desde el contenedor de base de datos

```
$ curl -s http://localhost:8080/api/tareas
{
    "total": 5,
    "tareas": [
        {
            "id": 1,
            "titulo": "Crear repositorio en GitHub",
            "descripcion": "Inicializar el repositorio remoto del proyecto y
                            definir ramas main y develop",
            "estado": "completada",
            "responsable": "Equipo",
            "creada_en": "Sat, 07 Nov 2026 15:08:35 GMT"
        },
        {
            "id": 2,
            "titulo": "Construir contenedor de aplicacion",
            "estado": "completada", ...
        },
        ...
    ]
}
```

10.6 Escritura: el contenedor web inserta en el contenedor db

```
$ curl -s http://localhost:8080/api/tareas -X POST \
       -H "Content-Type: application/json" \
       --data '{"titulo":"Entregar evidencias de la Semana 3", ...}'
{
    "id": 6,
    "titulo": "Entregar evidencias de la Semana 3",
    "descripcion": "Adjuntar capturas y documento en el aula virtual",
    "estado": "pendiente",
    "responsable": "Equipo",
    "creada_en": "Sat, 07 Nov 2026 15:09:21 GMT"
}
```

El registro se verifica a continuación directamente dentro del contenedor de base de datos:

```
$ docker compose exec db psql -U ciuser -d tareasdb \
       -c "SELECT id, titulo, estado FROM tareas ORDER BY id;"

 id |                  titulo                   |   estado
----+-------------------------------------------+-------------
  1 | Crear repositorio en GitHub               | completada
  2 | Construir contenedor de aplicacion        | completada
  3 | Construir contenedor de base de datos     | completada
  4 | Verificar comunicacion entre contenedores | en progreso
  5 | Preparar entrega de la semana 3           | pendiente
  6 | Entregar evidencias de la Semana 3        | pendiente
(6 rows)
```

La tarea creada mediante la API desde el contenedor web aparece almacenada en el contenedor de base de datos. Queda demostrada la comunicación en ambos sentidos: lectura y escritura.

10.7 Aislamiento del contenedor de base de datos

```
$ nc -z -w 2 localhost 5432        (desde la maquina anfitriona)
```

```
  -> conexion rechazada: el puerto 5432 no esta publicado

$ docker compose exec web python    (desde el contenedor web)
  -> conexion TCP establecida con db:5432 desde el contenedor web
```

PostgreSQL no es alcanzable desde la máquina anfitriona, pero sí desde el contenedor web. Esta es la prueba más concluyente de que el tráfico viaja por la red interna de Docker, de contenedor a contenedor.

10.8 Pruebas unitarias ejecutadas dentro del contenedor

```
$ docker compose exec web python -m pytest tests/ -v
============================= test session starts ==============================
platform linux -- Python 3.12.16, pytest-8.2.2, pluggy-1.7.0
rootdir: /usr/src/app
collected 3 items

tests/test_app.py::test_endpoint_inicio_responde              PASSED   [ 33%]
tests/test_app.py::test_host_base_datos_es_el_servicio_docker PASSED   [ 66%]
tests/test_app.py::test_crear_tarea_sin_titulo_devuelve_400   PASSED   [100%]

============================== 3 passed in 0.07s ===============================
```

10.9 Registros de los contenedores

```
$ docker compose logs --tail=6 db
ci_db | PostgreSQL init process complete; ready for start up.
ci_db | LOG:  starting PostgreSQL 16.15 on aarch64-unknown-linux-musl
ci_db | LOG:  listening on IPv4 address "0.0.0.0", port 5432
ci_db | LOG:  database system is ready to accept connections

$ docker compose logs --tail=6 web
ci_web | [INFO] Starting gunicorn 22.0.0
ci_web | [INFO] Listening at: http://0.0.0.0:5000
ci_web | [INFO] Booting worker with pid: 7
ci_web | 172.18.0.1 - - "GET /health HTTP/1.1" 200
ci_web | 172.18.0.1 - - "GET /api/tareas HTTP/1.1" 200
ci_web | 172.18.0.1 - - "POST /api/tareas HTTP/1.1" 201
```

11. Integración continua (pipeline en GitHub Actions)

Aunque la implementación de Jenkins corresponde a la Entrega 2, en esta primera entrega se dejó configurado un pipeline en GitHub Actions que valida automáticamente el trabajo realizado. El pipeline se ejecuta en cada push y en cada pull request sobre las ramas main y develop, y consta de dos trabajos encadenados:

Trabajo	Qué hace	Condición
pruebas	Instala las dependencias y ejecuta la suite de pytest	Siempre
contenedores	Construye las imágenes, levanta los dos contenedores y verifica que la	Solo si el trabajo anterior pasó

	comunicación entre ellos funcione	

.github/workflows/ci.yml

```
name: CI - Gestor de Tareas

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]

jobs:
  pruebas:
    name: Pruebas unitarias
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
      - run: pip install -r app/requirements.txt
      - working-directory: app
        run: python -m pytest tests/ -v

  contenedores:
    name: Construir y validar los dos contenedores
    runs-on: ubuntu-latest
    needs: pruebas
    steps:
      - uses: actions/checkout@v4
      - run: docker compose build
      - run: docker compose up -d
      - name: Verificar la comunicacion entre contenedores
        run: curl -fs http://localhost:8080/health | grep -q "establecida"
      - name: Apagar los contenedores
        if: always()
        run: docker compose down -v
```

De esta forma, ningún cambio puede fusionarse en las ramas principales si rompe las pruebas o si impide que los dos contenedores se comuniquen. Este es, en esencia, el principio de la integración continua.

12. Dificultades encontradas y soluciones

Dificultad	Causa	Solución aplicada
La aplicación no encontraba la base de datos al usar localhost	Dentro de un contenedor, localhost apunta al propio contenedor	Se usó el nombre del servicio (db) como host, resuelto por el DNS interno de la red red-ci
El contenedor web fallaba al arrancar la primera vez	PostgreSQL necesita unos segundos para inicializar la base de datos antes de aceptar conexiones	Se añadió un healthcheck con pg_isready y la condición depends_on / service_healthy, además de una política de reintentos en el código
La tabla no existía al consultar la API	La base de datos arranca vacía	Se montó el script db/init.sql en /docker-entrypoint-initdb.d/, que la imagen oficial ejecuta

		automáticamente en el primer arranque
Los datos se perdían al recrear el contenedor	El sistema de archivos de un contenedor es efímero	Se definió el volumen con nombre datos_postgres montado en /var/lib/postgresql/data
Riesgo de exponer credenciales en el repositorio	Las contraseñas estaban escritas en el archivo de configuración	Se trasladaron a variables de entorno, con env.example versionado y .env excluido mediante .gitignore
Imagen de la aplicación demasiado pesada	Uso de una imagen base completa y copia del código antes de instalar dependencias	Se usó python:3.12-slim, se reordenaron las capas del Dockerfile y se añadió .dockerignore

13. Roles y responsabilidades del equipo

Para esta entrega el trabajo se distribuyó de la siguiente manera:

Integrante	Rol	Responsabilidades en la Entrega 1
[Nombre]	Líder de repositorio	Creación del repositorio, estrategia de ramas, revisión de pull requests
[Nombre]	Desarrollo de la aplicación	API REST en Flask, pruebas unitarias
[Nombre]	Contenerización	Dockerfile del servicio web, docker-compose.yml, red y volúmenes
[Nombre]	Base de datos	Modelo de datos, script init.sql, configuración de PostgreSQL
[Nombre]	Documentación y evidencias	Recolección de evidencias, elaboración de este documento

Nota: ajustar esta tabla a los integrantes reales del subgrupo.

14. Conclusiones

- Se cumplió el objetivo de la entrega: el proyecto está alojado en un repositorio de GitHub y se construyeron dos contenedores Docker que se comunican entre sí, lo cual quedó demostrado con evidencias de ejecución real y no solo con la configuración.
- La comunicación entre contenedores no depende de direcciones IP sino del nombre de servicio, gracias a la resolución DNS que Docker habilita en las redes definidas por el usuario. Este mecanismo es lo que hace que el entorno sea reproducible en cualquier máquina.

- No basta con que un contenedor esté iniciado para que el servicio esté disponible. La combinación de healthcheck, depends_on con service_healthy y reintentos en el código resultó indispensable para que el arranque fuera fiable, que es un requisito para que un pipeline de integración continua no falle de forma intermitente.
- Mantener el contenedor de base de datos sin puertos publicados reduce la superficie de exposición y, al mismo tiempo, sirve como verificación de que la comunicación ocurre realmente por la red interna.
- La contenerización elimina la diferencia entre el entorno del desarrollador y el del servidor de integración, que es precisamente la condición que hace viable la integración continua y la base sobre la que se montarán Jenkins y Travis CI en las entregas 2 y 3.

15. Referencias

Docker Inc. (2026). Docker documentation. https://docs.docker.com/

Docker Inc. (2026). Networking overview — User-defined bridge networks. https://docs.docker.com/network/

Docker Inc. (2026). Compose file reference. https://docs.docker.com/compose/compose-file/

Docker Inc. (2026). Best practices for writing Dockerfiles. https://docs.docker.com/develop/develop-images/dockerfile_best-practices/

Fowler, M. (2006). Continuous Integration. https://martinfowler.com/articles/continuousIntegration.html

GitHub Inc. (2026). GitHub Actions documentation. https://docs.github.com/actions

Humble, J. & Farley, D. (2010). Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley.

Pallets Projects. (2026). Flask documentation. https://flask.palletsprojects.com/

PostgreSQL Global Development Group. (2026). PostgreSQL 16 documentation. https://www.postgresql.org/docs/16/

Politécnico Grancolombiano. (2026). Guía del proyecto grupal: Proyecto de software basado en herramientas de integración continua. Módulo Énfasis Profesional I.